APELEVEL / TECHNICAL PUBLICATION

RAGE REALMZ

HOW APELEVEL BUILT WHISPER MARKET

& ADDRESSED SOLIDITY SECURITY RISKS

Professional engineering report on the creation of Whisper Market, Solidity vulnerabilities and exploit classes identified and mitigated, secure blockchain tooling, wallet integration, and the development handoff after 473 steps.



473	21 / 21	80002	STEP 474
STEPS COMPLETED	FOUNDRY TESTS PASSED	POLYGON AMOY CHAIN ID	NEXT CHECKPOINT

Checkpoint: October 27, 2026 Branch: feature/polygon-wallet-auth Deployment: Not yet performed

SECTION 01

Executive engineering summary

ApeLevel developed the Whisper Market as a contract-mediated resource economy, then tested safeguards against important Solidity risk classes rather than relying on game-interface restrictions alone. This report distinguishes safeguards actually implemented and tested from vulnerabilities that remain possible and require more work. The test environment has not been deployed.

ApeLevel concluded its RAGE REALMZ development session at Step 473, having established the foundation for an Ethereum-NFT-compatible multiplayer game with a Polygon-based transactional economy. The core achievement was not the step count itself, but a sequence of interdependent engineering resolutions involving network architecture, smart-contract construction, dependency security, wallet behaviour, deployment provenance, and access-controlled infrastructure.

The current testnet stack comprises three prepared smart contracts: RageRealmzTestTILT, RageRealmzResources, and WhisperMarketTreasury. The local Foundry suite passed 21 of 21 tests. Coinbase Wallet connectivity to Polygon Amoy was restored after identifying and addressing an unrecognized-network error; access to the user’s Ethereum NFT skins remained functional.

The project also discovered a high-severity issue in a development dependency and responded by adopting verified native Solidity and Foundry tooling rather than retaining the unnecessary npm compiler wrapper. A separate deployment server and several authentication layers were prepared, but its default-deny restriction remains active.

Critical distinction: none of the three contracts has been deployed to Polygon Amoy during this session. No Coinbase Wallet deployment signature, token deposit, resource mint, or treasury transaction has occurred. The deployment user interface is still disabled.

NEXT EXECUTION POINT: STEP 474 — integrate session-bound CSRF protection and complete controlled browser access before enabling wallet-signed transactions.

SECTION 02

Architecture: Ethereum identity meets Polygon transactions

RAGE REALMZ is designed to let players connect on Polygon while retaining access to character skins controlled by NFTs on Ethereum Mainnet. The user’s EVM wallet address is the point of correspondence between networks. After proving wallet control through a verified signature, the game can query Ethereum NFT ownership read-only while reserving Polygon for game-currency activity.

This avoids requiring a player to change their wallet network simply to check eligibility for an NFT character skin. It also avoids moving the existing Ethereum NFTs onto Polygon. Ownership verification, wallet authentication, and economic authority remain logically separate.

Network roles

- Ethereum Mainnet: read-only RAGE QUITTERS NFT ownership and eligibility queries.
- Polygon Mainnet: existing UNINSTALL ($TILT) asset; read-only balance queries may inform a later testnet mirror.
- Polygon Amoy Testnet, chain ID 80002: contract deployment, mock tTILT, resource operations, and test-marketplace transactions.
- Game server: authoritative multiplayer state and validated session handling; it must not be mistaken for blockchain ownership authority.

A development-wallet designation is not a substitute for wallet-controlled signatures, and an NFT holder is not automatically entitled to privileged contract permissions.

SECTION 03

Creation of Whisper Market: contracts and trading rules

The proposed Whisper Market economy uses atomic blockchain transfers to reconcile resource ownership and consideration in a single transaction. A successful buy should transfer payment and resources together, while an unsuccessful operation should revert rather than create a partial purchase.

Contract responsibilities

- RageRealmzTestTILT: testnet ERC-20 token used for test transactions, not interchangeable with live Polygon Mainnet TILT.
- RageRealmzResources: ERC-1155 game-resource contract, supporting distinct resource identifiers and restricted minting.
- WhisperMarketTreasury: handles pricing, authorized deposits, resource exchanges, and treasury accounting constraints.

Initial Water-resource settings

- Treasury buy price: 1 test TILT per unit of Water.
- Treasury sellback payout: 1.25 test TILT per Water under the test configuration.
- Per-transaction purchase maximum: 100 Water.
- Lifetime treasury purchase-spend cap: 1,000 test TILT, under the locally tested logic.

The specified buy price of 1 tTILT and sellback payout of 1.25 tTILT create a potential loss-making round trip for the treasury if repeated by the same participant. This is an explicit economic risk requiring a design decision before unrestricted public use. Local functional tests do not establish solvency or sustainable economics.

SECTION 04

Smart-contract threat model and 21-test milestone

ApeLevel incorporated restrictive role permissions, non-reentrant transaction handling, and validation of treasury resource-receiver behaviour. These measures are intended to reduce the risks of unauthorized minting and deposits, unexpected token receivers, reentrant calls, and half-completed resource trades.

The Foundry suite executed 21 local EVM tests successfully. The recorded scenarios included access authorization and rejection, deposit handling, mint permissions, priced resource trades, invalid transfers, receiver reentrancy, Water transaction bounds, and the lifetime purchase-spend limit.

A passing test establishes the specified behaviour for its tested assumptions. It is not a formal proof of safety, a completed independent audit, or evidence that live Amoy transactions have succeeded. Tests must be expanded for unusual ERC-20 implementations, adverse liquidity, deployment configuration, privileged role transfers, and end-to-end game behaviour.

Confirmed result

21 / 21 local Solidity tests passed. Live deployment and integration testing remain outstanding.

SECTION 4A

Solidity vulnerabilities recognized and mitigated

The development session addressed risk classes common to smart-contract systems, not a confirmed exploit of ApeLevel code. The evidence is implementation decisions and passing local Foundry tests. The team did not discover an undisclosed zero-day Solidity compiler vulnerability or demonstrate that all risks are eliminated.

1. Reentrancy through hostile contract callbacks

When a treasury transfers ERC-1155 resources to a contract receiver, that receiver can execute callback code. If the treasury updates accounting in the wrong order, a hostile callback could attempt to initiate another operation before the first one finishes. ApeLevel introduced non-reentrant guards and tested a reentrant receiver scenario. The reported local test succeeded; hostile contract combinations beyond the tested receiver should still be examined.

2. Unauthorized minting and privileged treasury activity

An exposed resource mint or deposit path could undermine scarcity and balances if callers can bypass the intended roles. ApeLevel separated privileged actions and incorporated access checks using OpenZeppelin authorization primitives. Foundry tests exercised both authorized and unauthorized mint/deposit behaviours. Review of final role assignment, deployer privileges and any future administration changes is still needed.

3. Unsafe or unsolicited resource transfers

ERC-1155 receivers can accept or reject token deliveries through standard callback interfaces. An overly permissive receiver could admit unrelated resources or create accounting ambiguity. The treasury employed gated receiver behaviour, and the tests included invalid transfer conditions. Acceptance should remain limited to expected token contracts and transaction contexts.

4. Partial settlement, inconsistent balances, and transfer ordering

If resource movement and token consideration are separated into unrelated application actions, a failure can leave one party paying without receiving the corresponding asset. Whisper Market was designed so ERC-20 payment and ERC-1155 resource movement occur atomically within a blockchain transaction. Reverts roll back the whole on-chain operation. The game must nevertheless wait for confirmed receipts and must not equate an optimistic interface message with final settlement.

5. Transaction caps and economic abuse

ApeLevel incorporated a 100-Water per-transaction limit and a fixed 1,000-test-TILT lifetime purchase-spend cap into the locally tested design. Such constraints mitigate specified oversized purchases but do not automatically prevent splitting across wallets, role misuse, treasury depletion, or game-economic exploits. Any wallet-based limits should be evaluated against multi-wallet behaviour.

6. No unconstrained emergency withdrawal or upgrade path

The recorded treasury design avoided an arbitrary withdrawal function and unrestricted administrator upgrade path. This narrows privileged attack surfaces, though it also means contract errors or stranded funds may be harder to recover. A precise source-level review is required before drawing conclusions about every available privilege.

7. Important remaining economic vulnerability: buy/sell spread

In the test configuration, players buy Water for one tTILT and sell back for 1.25 tTILT. If the same actor can repeat that round trip against a funded treasury without other constraints, the rule creates a potential arbitrage cycle that extracts 0.25 tTILT per unit. This is a design risk, not a demonstrated exploit, and it should be resolved or intentionally constrained before unrestricted live trades are enabled.

What the evidence establishes

The 21 passing Foundry tests support the specifically exercised code paths. They do not rule out access-control misconfiguration at deployment, fee-on-transfer token incompatibilities, denial of service, front-running, chain reorganization, oracle assumptions, integer edge cases, or novel exploit compositions. No independent contract audit is recorded.

SECTION 05

The dependency-security investigation

During setup, ApeLevel identified a high-severity security advisory in the tmp dependency brought in by npm-based Solidity compiler tooling. The cited advisory was GHSA-7c78-jf6q-g5cm, with a reported CVSS score of 8.2. This was a development-supply-chain concern; the session did not establish that game contracts were compromised or that an exploit had occurred.

Rather than adding layers of tooling around the problem, ApeLevel removed the unnecessary vulnerable compiler wrapper and switched to a native Solidity compiler. The compiler binary and native Foundry release were integrity checked and their provenance examined. Foundry was configured to invoke pinned Solidity 0.8.38 in offline mode with optimizer settings of 200 runs.

The recorded environment used Foundry 1.8.6 and OpenZeppelin contract modules. The relevant dependency audit subsequently returned no reported vulnerabilities at that point. That result should be retained as time-specific evidence, not interpreted as a permanent assurance.

Why the decision mattered

The project reduced unnecessary transitive dependencies, improved build predictability, and avoided adding Hardhat solely to deploy the contracts. The existing viem installation supplies transaction-encoding and wallet-client functionality, including the correctly named polygonAmoy chain export.

SECTION 06

Wallet outage: Coinbase Wallet and the Amoy network

The game later encountered a connection failure when users selected Coinbase Wallet. Rather than assuming that the entire login system had broken, ApeLevel verified that the game server and authentication challenge endpoint were responding correctly. Investigation then focused on the wallet network-selection process.

The decisive error message was: “Unrecognized chain ID. Try adding the chain using addEthereumChain first.” The connector had attempted to switch to Polygon Amoy without reliably handling wallets that had never added the network.

ApeLevel added a fallback to wallet_addEthereumChain when the provider reported an unknown chain, supplying the Polygon Amoy chain identifier and network details. The code also checked the resulting wallet chain ID. After syntax validation, the user reported that Coinbase Wallet connected successfully on the testnet and NFT skins remained accessible.

This resolution preserved the intended separation: the wallet interacted on Polygon Amoy while Ethereum remained a read-only source for the user’s existing NFT skins. Future regression tests should cover users who have never added Amoy, declined network permission, changed accounts, and switched networks during an active session.

SECTION 07

Deployment artifact provenance and readiness

ApeLevel verified that the installed viem library exposes encodeDeployData, createWalletClient, and custom provider transport functionality. No additional blockchain framework was needed for the intended wallet-signed deployment workflow.

The TEST TILT deployment artifact was examined for contract name, target chain, ABI, constructor arguments, and Solidity metadata. RageRealmzTestTILT requires zero constructor arguments. Its encoded deployment bytecode was 3,710 bytes. The locally compiled bytecode and the prepared deployment artifact matched exactly.

Verified Keccak-256 deployment bytecode hash

0xfd9cc75a62c974adc952f2bc3267989672a6cf06eb420e7d39ee08dab4e88847

The artifact identifies Solidity 0.8.38 and OpenZeppelin ERC-20 / AccessControl dependencies. This is evidence of local artifact consistency, not a security attestation. A deployed address must be independently confirmed from an Amoy receipt and checked against the expected bytecode.

Authorized primary test wallet

ragequitters.eth — 0x486735D3dEfd63F4759f1d88df4E11e2Ec4698eB

The wallet was reported connected on Amoy and had previously been observed holding approximately 0.1 testnet POL. Neither observation constitutes an executed deployment.

SECTION 08

Isolation of the deployment infrastructure

The public RAGE REALMZ game runs through the existing server on port 3000. Rather than enabling deployments within that surface, ApeLevel created a separate workspace at scripts/contracts/amoy-deploy and a Node HTTP server bound to 127.0.0.1:4177. Its initial and current behaviour remains HTTP 403 for all requests, with the administrative page deliberately disabled.

Early curl tests returned HTTP 000 because the listener was no longer running; terminal history showed one manual Ctrl+C stop. A controlled start-request-stop test returned HTTP 403 and confirmed the implementation was operational. The project then checked Codespaces forwarding: port 3000 appeared with private visibility, and port 4177 was not forwarded. A direct test of a deployment source path through the public game server returned 404.

These tests support isolation of the inspected routes at the time of checking. They do not provide exhaustive proof that a future port-forwarding change, alternative route, or deployment configuration cannot expose the interface.

HTTP safeguards verified

- No-store caching and X-Content-Type-Options: nosniff.
- X-Frame-Options: DENY, Referrer-Policy: no-referrer, restrictive initial Content Security Policy.
- Origin-header rejection for unrelated origins; permitted origin still reaches default-deny handling.
- Explicit access-token validation preceding browser-session validation.

SECTION 09

Authentication controls added in Steps 449-473

A 256-bit random deployment access token was generated, saved with owner-only 0600 permissions, excluded from Git, and validated with a timing-safe comparison. Requests lacking the token were rejected. A correct token alone did not enable the page; it only progressed to the next locked layer.

A separate 256-bit session secret was created for HMAC-SHA256 browser sessions. Session expiration was configured for fifteen minutes. The session format was extended with a random identifier so two sessions created near-simultaneously do not share identical credentials.

In-memory individual-session revocation was developed and tested. A separate protected generation file was added to allow global invalidation of previously issued sessions, including when the server restarts. Individual revocation records still disappear after restart; generation invalidation is a global control and does not solve persistent per-session revocation.

An HttpOnly, SameSite=Strict session-cookie helper was written, with a 900-second Max-Age. Tests covered missing, malformed, duplicate, and valid cookies. The deployment server was updated to require both a deployment token and a valid cookie before reaching its disabled interface. All four recorded dual-authentication tests passed.

A local session-issuance helper was created and tested. Step 473 then created a CSRF module with cryptographically random tokens and timing-safe comparison. The CSRF module was syntax checked but not integrated into any server route. Likewise, a safe browser session-bootstrap flow has not been completed.

Security limitations to resolve

- Normal browser navigation cannot automatically provide the custom deployment-token header. An explicit, private session-bootstrap design is still required.
- The cookie currently lacks the Secure attribute while operating on localhost HTTP; establish the correct trusted-local / HTTPS access model before exposing functionality.
- CSRF tokens must be session-bound and verified for any state-changing operation.
- Origin headers can be absent or forged by non-browser clients and must not substitute for authentication.
- The public browser must never receive a wallet private key, a long-lived access token, or unconstrained signing authority.

SECTION 10

Confirmed versus outstanding work

Confirmed in this session

- Coinbase Wallet connects on Polygon Amoy; Ethereum-linked NFT skins remained visible after the fix.
- Three contract sources compiled; 21/21 recorded local Foundry tests passed.
- Native Solidity / Foundry toolchain established after dependency remediation.
- TEST TILT constructor, compiled metadata, bytecode size, and artifact hash examined.
- Private deployment server verified operational and default-deny; local isolation and Codespaces visibility inspected.
- Token, cookie, signed-session, generation-invalidation, and local helper tests passed.

Not yet completed

- Browser login/bootstrap and session-bound CSRF enforcement.
- Authenticated Coinbase Wallet deployment controls with explicit chain/account/transaction review.
- Deployment of TEST TILT, resources, or WhisperMarketTreasury to Amoy.
- Minting 100 Water to the intended test wallet, funding the treasury, or executing a live test buy/sell.
- On-chain treasury-balance display and resource inventory integration into RAGE REALMZ.
- Mainnet TILT read-only balance-based tTILT mirror and NFT-holder testing allocations.
- Expanded adversarial testing, economic review, contract address persistence, security audit, and production-release evaluation.

SECTION 11

Operational handoff: resume at Step 474

The project is paused on Git branch feature/polygon-wallet-auth in the working directory /workspaces/ragequitterz-game. The relevant development workspace is scripts/contracts/amoy-deploy. The files in scope include server.mjs, session.mjs, cookies.mjs, issue-session.mjs, csrf.mjs, index.html, and the Git-ignored credential files.

Step 474: review the current worktree and integrate CSRF protection into the private deployment authentication flow, ensuring that the CSRF token belongs to an authenticated session and is enforced on any state-changing endpoint. Keep default-deny behaviour until all required checks are in place.

The next milestone is to finalize a secure browser-session bootstrap without placing secrets in frontend code or URL parameters. Carefully resolve localhost-versus-Codespaces forwarding, TLS, Secure cookie behaviour, and any authorization state transitions.

Only after the access design is validated should the interface detect Coinbase Wallet, verify the connected account is the authorized test wallet, confirm Polygon Amoy chain ID 80002, present the exact deployment transaction for human review, and request a deliberate signature.

Following a successful Amoy receipt, record the actual deployed contract address and inspect the runtime code. Deploy the resources contract next, then the treasury with validated token and resource addresses. Once live, test authorized tTILT funding, allocation of 100 Water, atomic resource trades, and browser-game treasury views using receipt-backed confirmations.

Safe pause checklist

- Do not forward or expose deployment port 4177.
- Do not copy access-token, session-secret, generation values, or any wallet recovery material into logs or Git.

- Check git status; carefully review changes before making a development checkpoint commit.
- Continue one command at a time; preserve the Step 474 counter and avoid repeating completed tests unless needed for regression.

Engineering conclusion

Across 473 recorded steps, ApeLevel designed and locally tested Whisper Market while identifying and addressing Solidity risk classes including reentrancy, authorization failures, unsafe resource reception, partial settlement, and unbounded trades. It separately removed a vulnerable development-tool dependency, restored Coinbase Wallet compatibility, and verified the TEST TILT deployment artifact. The remaining priorities include economic-arbitrage review, authenticated browser deployment controls, and the first authorized Polygon Amoy deployment.

Release classification: pre-deployment testnet engineering. No live deployments or signed deployment transactions are asserted in this report.



RAGE REALMZ
Character identity and marketplace architecture remain distinct: the character shown is brand artwork, not evidence of a minted testnet resource or deployed contract.

